ARTÍCULO ENVIADO

¿Cómo es que Netflix lee mi mente?

José E. Rojas-Macedo y Ángel Hernández-Castañeda

Introducción

Dentro de la era digital, la personalización de servicios se ha convertido en una característica esencial para ofrecer una experiencia de usuario (UX) superior. Desde las plataformas de streaming, hasta algunos *e-commerce*, utilizan sistemas de recomendación que nos ofrecen contenido, productos o servicios basados en nuestros gustos. ¿Cómo se logra esto? La respuesta está en el uso de algoritmos de recomendaciones.

Uno de los más utilizados, es el filtrado colaborativo (FC), una técnica que permite que plataformas como Netflix, Amazon o Spotify ofrezcan contenido basado en lo que personas con gustos similares han disfrutado. En este artículo, se explorará una aproximación simple de cómo se construyen estos sistemas, de qué forma se entrenan y cómo se pueden desarrollar usando Python y la librería de Surprise.

El algoritmo detrás de las recomendaciones

Un algoritmo representa una secuencia de instrucciones precisas que permiten resolver un problema o ejecutar una tarea específica, como la receta para preparar un pastel. En los sistemas de recomendación, el algoritmo recibe como entrada un conjunto de datos, en este caso, puede incluir las películas que has visto y las valoraciones que has dado sobre el contenido. Con dicha información, el algoritmo busca patrones entre tus datos y los de miles o millones más para sugerir nuevos elementos.

En un nivel básico, el FC cuenta con dos enfoques o métodos:

1. **Basado en usuarios:** Encuentra usuarios que tengan preferencias similares a un usuario para recomendarle películas que estos hayan disfrutado.
2. **Basado en elementos:** Encuentra películas que tengan características similares a las que un usuario ha disfrutado para recomendarle otras similares.

En ambos casos, los algoritmos buscan patrones en los datos que pueden ser aprovechados para generar recomendaciones acertadas. Durante los últimos años, el filtrado colaborativo ha evolucionado con la integración de técnicas más avanzadas como la **Descomposición de Valores Singulares** (SVD) o **Factorización de Matrices** (MF), que permiten mejorar la precisión de las recomendaciones.

Para este artículo, se utilizó el enfoque basado en usuarios, el cual, como se mencionó, analiza las preferencias de los usuarios para determinar qué películas sugerir a cada persona. La ventaja de usar este enfoque es que, a diferencia del basado en elementos, este se basa en la experiencia de otros usuarios para hacer recomendaciones (ver Figura 1).

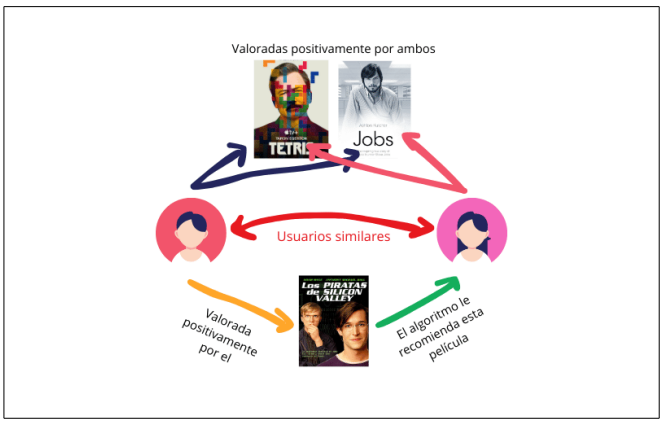

Figura 1. Funcionamiento del filtrado colaborativo basado en usuarios

El filtrado colaborativo es un tipo de algoritmo que puede conocerte mejor de lo que te conoces a ti.

Metodología

Esta sección describe cómo se construyo el sistema de recomendación utilizando Python y la librería de Surprise:

1. **Importación de datos:** Se utiliza un conjunto de datos de películas que incluye información sobre los usuarios, películas y valoraciones de los usuarios por película.
2. **Preprocesamiento de datos:** Los datos son limpiados y transformados en un formato adecuado para el entrenamiento del modelo. Para este proceso se usan las librerías de Pandas y Numpy.

3. **Entrenamiento del modelo:** Con la biblioteca Surprise se desarrolla el modelo, para ello, se utiliza la técnica de SVD. Este algoritmo descompone la matriz de las valoraciones de los usuarios en factores que representan las características de los usuarios y de las películas, para así generar recomendaciones más precisas.

4. **Evaluación del modelo:** El rendimiento del modelo se evalúa utilizando la métrica *RMSE (Root Mean Squared Error)*, la cual, mide la diferencia entre las predicciones del modelo y las valoraciones reales de los usuarios.

5. **Generación de recomendaciones:** Se generan recomendaciones para un usuario específico mediante una función que manda a llamar al modelo de filtrado colaborativo.

Importación de datos

Como primer paso para la construcción de un sistema de recomendación, es necesario disponer de un conjunto de datos adecuado para el análisis y entrenamiento del algoritmo.

En este sentido, se utilizó un conjunto de datos de películas que incluye información sobre los usuarios, películas y valoraciones de los usuarios por película. El conjunto es una versión ligera de toda la base de datos de *MovieLens*, una plataforma de recomendación de películas, que incluye alrededor de 100,000 valoraciones de 600 usuarios aplicadas a 9,000 películas hasta 2018 y que cuenta con alrededor de 3,600 etiquetas, compuesto de 4 archivos CSV: *ratings.csv*, *movies.csv*, *links.csv* y *tags.cvs*, de los cuales se utilizaron solamente los dos primeros.

El conjunto de datos de MovieLens apareció por primera vez en 1998, esta es una plataforma de recomendación que solicita a sus usuarios realizar valoraciones a distintas películas, y así proporcionarles recomendaciones personalizadas (ver Figura 2); y describen las preferencias de los usuarios en cuanto a películas, los datos producidos toman forma de tupla con 4 datos principales: usuario, ID de la película, valoración (en escala de 0-5 estrellas) y marca de tiempo [Harper and Konstan, 2015].

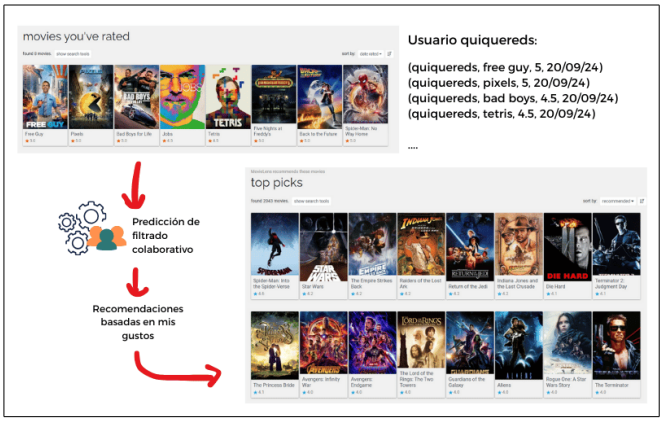

Figura 2. Plataforma de MovieLens utilizando un sistema de recomendación

Para este punto, se usó la librería de Pandas, la cual es una herramienta de código abierto diseñada para la manipulación y análisis de datos usando Python, de esta forma puede utilizarse este lenguaje para cargar, alinear, manipular e incluso fusionar datos. Como primer paso, se cargaron los datos de los archivos CSV y se almacenaron en un DataFrame de Pandas para su posterior procesamiento. En el Cuadro 1 y 2 se muestra un ejemplo de los datos de los archivos *ratings.csv* y *movies.csv*, respectivamente.

La visualización de los dataframes anteriores hace más sencilla la compresión de los tipos de datos que existen en el conjunto que se está evaluando, puede observarse que en el primero se muestra información de la película (ID, género y nombre de la película), mientras que en el segundo, las calificaciones que los usuarios han puesto sobre las películas (UserID, MovieID, calificación y estampa de tiempo).

MovieLens: una base de datos que ha sido empleada en la investigación de sistemas de recomendación.

Preprocesamiento de datos

El preprocesamiento de datos resulta bastante importante para asegurar que los datos estén dentro de un formato adecuado para el entrenamiento del modelo, y así garantizar que el algoritmo pueda aprender de ellos. En este caso, se realizó la limpieza de los datos y la transformación de los mismos en un formato adecuado para el entrenamiento del modelo.

El siguiente paso, es realizar una combinación de ambos dataframes para obtener un único dataframe que contenga toda la información necesaria para el entrenamiento del modelo. Para ello, se utilizó la función *merge* de Pandas, la cual permite combinar dos dataframes a través de una columna en común, en este caso, la colum-

userId	movieId	rating	timestamp
1	1	4.0	964982703
1	3	4.0	964981247
1	6	4.0	964982224
1	47	5.0	964983815
1	50	5.0	964982931

Cuadro 1: Dataframe de valoraciones

movieId	title	genres
1	Toy Story (1995)	Adventure—Animation—Children—Comedy—Fantasy
2	Jumanji (1995)	Adventure—Children—Fantasy
3	Grumpier Old Men (1995)	Comedy—Romance
4	Waiting to Exhale (1995)	Comedy—Drama—Romance
5	Father of the Bride Part II (1995)	Comedy

Cuadro 2: Dataframe de películas

na *movieId*. El resultado se muestra en la Tabla 3.

El siguiente paso, aunque puede verse un poco complejo, es tomar los datos de los usuarios, las películas y los géneros para convertirlos en una matriz de valores numéricos ordenados o binarios (0 y 1), es algo que resulta obligatorio para que una computadora pueda procesarlos de forma fácil en un sistema de recomendación.

Para ello, se utilizaron algunas clases provenientes de la librería Scikit-learn, tales como *LabelEncoder* y *MultiLabelBinarizer* para convertir los datos categóricos en valores numéricos y binarios, respectivamente. El resultado se muestra en la Tabla 4.

Con los datos clasificados de esta nueva forma, se procedió a dividir el conjunto de datos en dos partes: una para entrenar el modelo y otra para evaluar su rendimiento. Para ello, se utilizó la función *train_test_split* de Scikit-learn, la cual divide el conjunto de datos en dos partes, una para entrenamiento y otra para prueba, en este caso, se utilizó una proporción de 80 % para entrenamiento y 20 % para prueba.

Entrenamiento del modelo

Para el entrenamiento del modelo, se utilizó la librería de Surprise, la cual es una biblioteca de Python que proporciona una serie de algoritmos de recomendación listos para usar. En este caso, se utilizó el algoritmo SVD, el cual es una técnica de factorización matricial que descompone la matriz de valoraciones de los usuarios.

La implementación del modelo se muestra a continuación:

```
from surprise import Dataset, Reader
from surprise import SVD
from surprise.model_selection import cross_validate

# Cargar el conjunto de datos
reader = Reader(rating_scale=(0.5, 5))
data = Dataset.load_from_df(df[['userId',

# Obtener el conjunto de entrenamiento
trainset = data.build_full_trainset()

# Entrenar el modelo
model_svd = SVD()
model_svd.fit(trainset)
```

En el código anterior, se carga el conjunto de datos en un objeto de tipo *Dataset* de Surprise, el cual es necesario para el entrenamiento del modelo. Posteriormente, se crea una instancia del modelo SVD y se entrena con el conjunto de datos.

Evaluación del modelo

La evaluación del modelo es una parte fundamental para determinar su rendimiento y precisión en la generación de recomendaciones. En este caso, se utilizó la métrica *RMSE* para evaluar el rendimiento del modelo. Esta métrica mide la diferencia entre las predicciones del modelo y las valoraciones reales de los usuarios, y se calcula como la raíz cuadrada de la media de los errores al cuadrado (Eq 1).

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1}$$

Donde:

- y_i es la valoración real del usuario i.
- $\hat{y}_i$ es la predicción del modelo para el usuario i.
- n es el número total de usuarios.
- $\sum$ es la suma de los errores al cuadrado.
- $\sqrt{}$ es la raíz cuadrada.

userId	movieId	rating	title	genres
1	1	4.0	964982703	Adventure—Animation—Children—Comedy—Fantasy
1	3	4.0	964981247	Comedy—Romance
1	6	4.0	964982224	Action—Crime—Thriller
1	47	5.0	964983815	Mystery—Thriller
1	50	5.0	964982931	Crime—Mystery—Thriller

Cuadro 3: Dataframe combinado de ratings y movies

- $\frac{1}{n}$ es el promedio de los errores al cuadrado.

El resultado de la evaluación del modelo fue de 0.475, lo que se traduce a que el modelo se está equivocando en promedio 0.475 estrellas en sus predicciones (recordemos que la escala de valoración es de 0 a 5 estrellas), lo cual es una buena métrica, ya que el error es relativamente bajo.

Referencias

[Harper and Konstan, 2015] Harper, F. M. and Konstan, J. A. (2015). The movielens datasets: History and context. ACM Trans. Interact. Intell. Syst., 5(4).

SOBRE LOS AUTORES

José Enrique Rojas Macedo recibió el título de Ingeniero de Software por la Universidad del Valle de México (UVM) en 2023, especializado en el desarrollo de aplicaciones móviles con Flutter, Google Cloud y Firebase. Motivado por empujar constantemente sus límites, contribución a causas sociales, compartir conocimiento y actualmente estudiando la Maestría en Ciencias de la Computación en la Universidad Autónoma del Estado de México. Sus intereses en investigación son el análisis de sentimientos, reconocimiento de patrones y minería de datos.

Ángel Hernández Castañeda recibió su Maestría y Doctorado en Ciencias de la Computación, con honores, por el Centro de Investigación en Computación (CIC) del Instituto Politécnico Nacional (IPN), en 2013 y 2017, respectivamente. Asimismo, es autor de diversas publicaciones en revistas internacionales de alto impacto. Actualmente es Profesor Investigador de Tiempo Completo en la Universidad Autónoma del Estado de México y miembro del Sistema Nacional de Investigadores (SNI) de México. Sus intereses en investigación incluyen el procesamiento de lenguaje natural, la minería de datos, el aprendizaje automático y el reconocimiento de patrones.